

Dream Machine

System Reference - every function, feature, and what we actually use

An automated algorithmic trading system for NSE Indian equities. It runs two sleeves (intraday + swing), a research engine that generates and certifies strategies (the Strategy Maker), a full backtest/validation stack, a regime router, live broker execution via Zerodha Kite, a paper-trading simulator with a shadow book, a web dashboard, and Telegram control.

This document inventories the whole codebase: what each module does, which ones are used the most, and which features are built but not currently needed.

At a glance

~50 src modules | 20 Strategy-Maker modules | in-house indicator library | 868 passing tests | React + FastAPI web app.

Live status: SWING sleeve active (daily CNC, held overnight); INTRADAY sleeve suspended (no edge found). Paper mode at Rs.10L virtual capital.

Legend: LIVE = runs in the production loop SWING = the active edge sleeve DATA = market-data infrastructure RESEARCH = backtest/strategy-discovery OPS = CLI/operations DORMANT = built + tested but wired into nothing

1. How the system fits together

The bot is organized as concentric layers. From the outside in:

- CLI / ops (bot.py) - one entry point: auth, run, backtest, validate, sweep, tune, make, status, golive/gopaper, shadow.
- Live loop (main.py) - the once-per-cycle engine: fetch data -> build candles -> scan -> risk-check -> execute -> reconcile. Runs the intraday scanner+router AND, once a day near the close, the swing sleeve.
- Two sleeves - INTRADAY (MIS, 5-min, scanner+regime router) is SUSPENDED (no validated edge). SWING (daily, CNC, held overnight) is the ACTIVE money-making path.
- Strategy Maker (maker/) - the research engine that invents strategies from a block grammar and certifies them through a screen -> gauntlet -> sealed reserve funnel before any of them are allowed near capital.
- Backtest + validation stack - the honest event-driven backtester, walk-forward OOS, plateau/Monte-Carlo/slippage stress, and the go-live readiness gate.
- Data infrastructure - live WebSocket ticks, tick->candle builder, REST fetcher, the historical store, and the daily tradeable-universe builder.
- Surfaces - a React + FastAPI web dashboard and a Telegram controller for remote status/control.

The strategy lifecycle (research -> live)

generate (block grammar) -> cheap screen (kill ~90%) -> gauntlet (param sweep + walk-forward OOS at a rising acceptance bar) -> sealed single-shot reserve holdout -> ALIVE -> wired into the swing sleeve -> paper (Rs.10L) + shadow book -> go-live gate -> live CNC with exchange-side GTT stops.

2. The features we use the most

Ranked by how central they are to a normal trading day. These are the modules that run (or are touched) on essentially every cycle.

logger

Structured logging used by every module - the single most imported file in the codebase. [imported by 47 modules]

strategy

The signal engine: `generate_signal(symbol, df, cfg) -> TradeSignal`. The framework every strategy plugs into; also the ATR/SL/target and regime helpers. [imported by 11 modules]

swing_engine

THE active sleeve. Runs once a day: manages exits (ATR trail), scans conviction-ranked entries, capital-aware sizing with refusal logging, the shadow book, and live CNC+GTT execution with broker reconciliation. [imported by 3* modules]

auth

Daily Kite session load + token validation. Nothing online works without it. [imported by 8 modules]

data_fetcher / data_streamer / tick_candle_builder

The market-data spine: REST candles, the live WebSocket tick stream, and building candles from ticks with zero per-cycle REST. [imported by 7/2/2 modules]

trade_db / trade_ledger

The SQLite ledger - every trade, signal, and routing decision (paper, live, and shadow) is recorded here. The source for all reports. [imported by 6/2 modules]

costs

The Zerodha cost model (intraday MIS + delivery CNC). Now deducted from every paper AND shadow P&L - the flat DP charge dominates small trades. [imported by 5 modules]

regime / market_state / router

The regime router: classify the market (trend/range/chop/quiet), then pick which edges are active. For the regime-agnostic winners it now drives the exposure throttle. [imported by 6/4/3 modules]

risk_manager / position_manager / order_executor

Live-trading guts: risk limits + kill switches, open-position bookkeeping, and Kite order placement (CNC entry + GTT OCO stop/target). [imported by 2/3/2 modules]

ops

Backs 'bot status'/'bot shadow'/mode-flip - the operational snapshot and the shadow report you read each evening. [imported by 5 modules]

universe_builder / scanner / volume_profile

Builds the daily tradeable universe (F&O-liquid NSE names) and ranks it; RVOL/volume-profile scoring. [imported by 2/2/4 modules]

** swing_engine has a low import count because only main.py wires it, but it is the single most important runtime component - it is where the actual edge trades.*

3. Full module reference

3.1 Live trading loop (runs in production)

`main.py` LIVE

The trading engine loop: startup wiring, per-cycle scan/execute, the once-a-day swing run, the EOD square-off + daily summary, the swing missed-run heartbeat watchdog, and clean shutdown.

`strategy.py` LIVE SWING

Signal framework: generate_signal + TradeSignal, indicator compute, EMA/supertrend helpers, ATR stop/target sizing, N-bar swing pivots, MTF gate, market_regime.

fn: generate_signal, market_regime, _atr, _sl_target, _swing_pivots

`swing_engine.py` SWING LIVE

The daily/positional sleeve. run_daily(): reconcile-with-broker (live), manage ATR-trailing exits with gap-through-stop simulation, conviction-ranked entries with capital-aware sizing + refusal logging + regime exposure throttle, the realistic-portfolio shadow book, net-cost accounting, and state persistence.

fn: run_daily, reconcile_with_broker, _manage_exits, _simulate_exit, _scan_entries, _net_pnl, _scan_shadow

`risk_manager.py` LIVE

Pre-trade risk gate: per-trade risk %, max positions, daily-loss and give-back-from-peak kill switches, spread + margin checks, market-open test.

`position_manager.py` LIVE

Open-position bookkeeping, trailing-stop state, and square-off timing.

`order_executor.py` LIVE

Kite order placement: market/limit entries, GTT OCO (stop+target), modify_gtt (self-ratcheting trail), cancel, and order-fill monitoring.

`paper_trader.py` LIVE

Simulated fills for paper mode (realistic LIMIT crossing + slippage) so paper mirrors live without real orders.

`live_router.py` LIVE

Positive allow-list of the intraday strategies the live loop may run (now suspended); keeps the swing sleeve separate.

`scanner.py` LIVE DATA

Intraday scanner: score + rank the universe each cycle by momentum/range-position/gap/RVOL; persists point-in-time rankings (shadow scan).

`alert_manager.py` / `telegram_controller.py` / `command_channel.py` LIVE OPS

Notifications and remote control: Telegram alerts + command polling, and an out-of-process command channel to pause/resume the loop.

`state_store.py` LIVE

Crash-recovery snapshot of loop state so a restart resumes cleanly.

`market_calendar.py` / `event_calendar.py` LIVE

Is the market open right now, and 'skip volatile event days' filtering (results, budget, etc.).

3.2 Data infrastructure

`data_fetcher.py` DATA LIVE

REST candle fetch from Kite with retry/rate-limit handling.

`data_streamer.py` DATA LIVE

Live KiteTicker WebSocket feed with auto-reconnect.

tick_candle_builder.py DATA LIVE

Builds OHLCV candles from streamed ticks - zero per-cycle REST once warm.

backtest_store.py DATA RESEARCH

The SQLite candle store (data_cache/backtest_data.db) for all backtests - multi-timeframe, delta-loaded.

historical_loader.py DATA RESEARCH

Chunked historical download (respects Kite's per-request day caps).

candle_cache.py DATA RESEARCH

Day-stamped CSV cache for backtest/sweep data.

universe_builder.py DATA LIVE

Builds the daily tradeable universe: F&O-liquid NSE names in a price band, auto-derived from the NFO instrument dump. filter_universe applies liquidity/volatility filters.

stock_selector.py DATA RESEARCH

Liquid-stock ranking for backtest universes (turnover + ATR% filters).

volume_profile.py / profile_store.py DATA LIVE

Volume profiles + time-of-day RVOL, and their persistence across sessions.

timeframe.py DATA RESEARCH

Pure OHLCV resampling (e.g. 15min -> daily).

3.3 Strategy Maker (the research / discovery engine)

maker/ generates candidate strategies from a constrained block grammar and certifies them through a leak-proof funnel. This is where new edges come from. Used heavily in research, not in the live loop.

grammar.py RESEARCH

The Candidate object + compile() -> a pure signal fn the existing backtester runs. Deterministic cid; enforces parsimony (<=3 conditions, <=4 tunable params). Holds all block evaluators (setups, triggers, exits, candle/session helpers).

blocks.py RESEARCH

The block library: 21+ setup/trigger/exit/regime/hold blocks, each rationale-enforced and sleeve-tagged (swing vs intraday).

generate.py RESEARCH

Random candidate sampling over the implemented block whitelist (swing + intraday), both long and short.

constraints.py RESEARCH

Generation-time rejects: parsimony, turnover/cost budget, short-on-CNC, duplicates, prior-falsified regions.

screen.py RESEARCH

The cheap screen: one fast backtest per candidate; kills on too-few-trades / low-PF / net-negative / outlier-carried / low-selectivity (breadth). Plus the shared walk-forward oos_metrics primitive.

fn: screen_candidate, screen_decision, oos_metrics, _breadth

run_gauntlet.py RESEARCH

The gauntlet: sweep a candidate's tunable axes, pick the in-sample best, then test it OOS against the rising acceptance bar.

reserve.py RESEARCH

The sealed single-shot holdout (RULE 2): each family gets ONE reserve exam ever; leak-proof (data unreadable outside evaluate_once). PASS requires PF>=bar, trades>=20, net>0, not outlier-carried, and selective (breadth).

fn: evaluate_once, effective_reserve_table, invalidate_reserve

registry.py / bar.py RESEARCH

Append-only trial registry (DB-trigger-enforced, RULE 1) + the rising acceptance bar pf_required(N)=1.2+0.15*log10(N/10).

[campaign.py](#) / [parallel_campaign.py](#) **RESEARCH**

End-to-end funnel drivers - serial and process-parallel (byte-identical results).

[survivorship.py](#) **RESEARCH**

NEW: audits the backtest universe for newcomer/partial-history bias (drops those names) and discloses the residual true-survivorship bias.

[cache.py](#) / [writer.py](#) / [vscreen.py](#) / [coherence.py](#) / [data_adjust.py](#) / [admission.py](#) **RESEARCH**

Support: indicator cache (reserve/screen-disjoint), single-writer registry queue, vectorized breakout screen (with a conservatism guarantee), timeframe-coherence gate, corporate-action adjustment, and paper/correlation admission.

3.4 In-house indicator library

[indicators/core.py](#) **RESEARCH**

Pinned indicator math (no TA-Lib): SMA/EMA/Wilder-RSI/ATR/ADX/MACD/Bollinger/BB-width/Donchian/stochastic, golden-fixture tested to 6dp.

[indicators/swings.py](#) / [fib.py](#) / [levels.py](#) **RESEARCH**

N-bar swing pivots with confirmation lag (no look-ahead), Fibonacci retrace/extension off confirmed pivots, and 4-tier support/resistance zones.

3.5 Backtest, validation & tuning

[backtester.py](#) **RESEARCH**

The honest event-driven backtest engine: ATR trailing / fixed-target exits, entry-window gating, slippage, delivery/intraday costs. BacktestResult + trade records.

[backtest_runner.py](#) / [backtest_run.py](#) **RESEARCH** **OPS**

Multi-strategy multi-timeframe orchestration + the matrix report.

[validation.py](#) **RESEARCH**

The 5-stage validation battery: Monte-Carlo, drop-best-trades, sub-period consistency, parameter plateau, cost/slippage stress, walk-forward, paper-vs-backtest, and the scorecard.

[param_sweep.py](#) / [sweep.py](#) **RESEARCH** **OPS**

Grid search + walk-forward parameter sweeps over the backtester.

[auto_tuner.py](#) / [autotune.py](#) **RESEARCH** **OPS**

Walk-forward tuner that writes a validated winner to the bounded overlay only if the OOS bar passes.

[overlay.py](#) **LIVE** **OPS**

The bounded, validated parameter sandbox applied at startup (hard bounds the tuner can never exceed).

[maker_certified.py](#) / [maker_gauntlet.py](#) **SWING**

The materialized manifests of reserve-certified edges and gauntlet survivors, wired into the swing sleeve as maker_/mkg_strategies.

[strategy_library.py](#) **SWING** **RESEARCH**

The hand-mined strategy implementations (donchian, bb, supertrend, vcb, VWAP family, etc.) - the origin set; most are now benched in favor of maker edges.

[strategy_meta.py](#) **SWING**

Loads strategies/*.yaml: regime validity + pre-validated param sets + regime_fit that the router reads.

3.6 Operations, gating & CLI

[bot.py](#) **OPS**

The unified lifecycle CLI: auth/run/backtest/validate/sweep/tune/make/status/shadow/golive/gopaper.

[go_live.py](#) / [ops.py](#) **OPS**

The paper-evidence readiness gate (never flips on its own) and the operational snapshot/shadow report.

[auth.py](#) / [make.py](#) / [validate.py](#) OPS

TOTP login, the Strategy-Maker CLI, and the validation-pipeline CLI.

[backfill_profiles.py](#) / [setup_autostart.ps1](#) / [start_bot.bat](#) OPS

Volume-profile backfill + Windows autostart/watchdog scripts.

3.7 Web dashboard (webapp/ + frontend/)

[webapp/](#) (FastAPI) OPS LIVE

A local dashboard: live status, positions/trades, strategy roster (validated vs benched), config editor, backtest jobs, log viewer, and a WebSocket live feed. Served on the laptop, reachable cross-device via Tailscale. React/Vite frontend built into webapp/static.

4. Features built but not needed (right now)

These are wired into NOTHING in the live path (0 production imports). They are tested scaffolding from earlier design directions. None of them run today. Options: keep as future scaffolding, or archive them to slim the codebase.

`regime_analyst.py` DORMANT

An LLM/analyst-style regime commentary scaffold. Left over from the AI phase that was deliberately removed; not in the deterministic path. The clearest archive candidate.

`mtf_replay.py` DORMANT

Multi-timeframe veto counterfactual replay (would the higher-TF gate have helped?). The MTF confirm gate itself is disabled (`mtf_confirm.enabled=false`), so this analysis is unused.

RECLASSIFIED - do NOT archive:

- `drift_audit.py` is NO LONGER dormant: it is now the overlay FREEZE guard. The tuner may not overwrite live parameters unless a passing `DriftVerdict` shows the winner beats baseline (`overlay.require_drift_audit=true`). It earns its place.
- `adaptive_params.py` / `adaptive_bounds.py` / `router_backtest.py` are the REGIME-adaptation modules. They look dormant because 'every certified edge is regime-agnostic' - but that is a SEARCH ARTIFACT, not a market fact: only 15.5% of generated candidates carry any regime block (the parsimony budget drops most), and the actual directional-regime block 'trend_side' appears in just 2.8%. The search barely tests regime, so 'regime doesn't help' is unproven. Keep these until the generator properly represents regime blocks and the question is answered honestly.

Also currently switched OFF (by design, not removable)

- INTRADAY sleeve (`intraday.enabled=false`) - suspended because no intraday edge survived validation on any timeframe. The scanner/live_router/supertrend/orb machinery is intact but idle.
- MTF confirmation gate (`mtf_confirm.enabled=false`) - never beat its own no-MTF baseline.
- strategy_library origin edges (bb, vcb, double_reversal, etc.) - mostly benched (`validated:false`) in favor of maker-certified edges; kept for reference/fallback.

Recommendation (revised after review)

Only TWO modules are safe to archive today: `regime_analyst.py` (AI-era leftover) and `mtf_replay.py` (analysis for a gate that is disabled). The other four that looked dormant were reclassified above: `drift_audit.py` is now the active overlay freeze-guard, and the three regime-adaptation modules must stay until the generator actually tests regime (today it barely does - see the 15.5% / 2.8% figures). Archiving prematurely would bury a finding the search hasn't earned the right to make. Everything else earns its place.

5. What you actually touch day to day

Every trading day	<code>cd trading-bot -> .venv activate -> 'python bot.py auth' -> 'python bot.py run'</code> (before ~3:00 PM). Stop with Ctrl+C.
Check status	<code>'python bot.py status'</code> (mode, token, market, paper P&L, go-live gate).
Read the shadow book	<code>'python bot.py shadow'</code> (what the refused/all signals would have done).
Research new edges	<code>'python bot.py make generate --max-trials N --symbols K' -> 'make status'</code> .
Backtest / validate	<code>'python bot.py backtest ...' / 'python bot.py validate ...'</code> .
Go live (gated)	<code>'python bot.py golive --confirm'</code> (readiness gate enforced) / <code>revert 'gopaper'</code> .
Web dashboard	<code>start_webapp.bat -> local FastAPI + React UI</code> (cross-device via Tailscale).

Bottom line: on a normal day you touch exactly two commands (auth, run) and read two reports (status, shadow). Everything under sections 3.3-3.5 is the research/validation machinery you run occasionally to find and prove new edges - not part of the daily loop. Section 4 is dead weight you can archive whenever you want a leaner codebase.